Backend Interview Questions

Section 1: Node.js Fundamentals — Interview Answers (1–20)

1. What is Node.js, and why is it used for backend development?

Node.js is a JavaScript runtime that runs JavaScript outside the browser. It is used to build APIs, web servers, real-time applications, and backend services.

2. How is Node.js different from JavaScript running in a browser?

JavaScript is the language; Node.js is a runtime. Browsers provide DOM and window APIs, while Node.js provides server-side APIs such as fs, http, and process.

3. Is Node.js single-threaded?

JavaScript normally runs on one main thread, but Node.js also uses OS-level asynchronous I/O and a libuv thread pool for certain operations. This lets it handle many concurrent requests efficiently.

4. Explain Node.js architecture: V8, libuv, event loop, and thread pool.

V8: Executes JavaScript.

libuv: Provides the event loop and asynchronous I/O support.

Event loop: Coordinates callbacks for asynchronous operations.

Thread pool: Handles certain tasks, such as file-system operations and cryptography.

5. What is the event loop, and why is it important?

The event loop allows Node.js to process asynchronous operations without blocking the main JavaScript thread while waiting for I/O. This improves concurrency and responsiveness.

6. Explain the phases of the Node.js event loop.

The main phases are timers, pending callbacks, idle/prepare, poll, check, and close callbacks. (Tpipcc) Each phase handles specific callbacks; Promise microtasks and process.nextTick() have separate scheduling behavior.

7. Difference between call stack, callback queue, and microtask queue?

Call stack: Executes synchronous JavaScript.

Callback queues: Hold callbacks waiting for their scheduled execution.

Microtask queue: Handles Promise reactions; Node.js also has a special process.nextTick() queue.

8. Difference between process.nextTick(), Promise.then(), setTimeout(), and setImmediate()?

process.nextTick(): Runs after the current operation, before the event loop continues.

Promise.then(): Runs as a Promise microtask.

setTimeout(): Runs after its minimum timer delay.

setImmediate(): Runs in the check phase, commonly after I/O polling.

Exact ordering can depend on context.

9. Difference between synchronous and asynchronous operations?

Synchronous: The current execution waits until the operation completes.

Asynchronous: The program can continue executing while the operation is pending, with its result handled later.

10. What is blocking versus non-blocking I/O?

Blocking I/O prevents the JavaScript thread from continuing until the operation completes. Non-blocking I/O lets it continue while waiting for the result.

11. What happens when a CPU-intensive task runs on a Node.js server?

It blocks the main JavaScript thread, delaying other requests and callbacks. Use worker threads, separate processes, or background jobs for expensive CPU work.

12. What are CommonJS and ES Modules?

They are JavaScript module systems.

CommonJS: Uses require() and module.exports.

ES Modules: Uses import and export.

ES Modules are the standard JavaScript module system; Node.js supports both.

13. Difference between module.exports and exports?

module.exports defines the value exported by a CommonJS module. exports is initially a reference to it.

module.exports = { getUser };

Reassigning exports alone does not change module.exports.

14. What is npm? Difference between dependencies, devDependencies, and peerDependencies?

npm manages packages and project scripts.

dependencies: Packages required at runtime.

devDependencies: Tools used mainly for development and testing.

peerDependencies: Packages expected to be provided by the consuming project, commonly for libraries and plugins.

15. Purpose of package.json and package-lock.json?

package.json: Defines project metadata, scripts, and dependency requirements.

package-lock.json: Records the resolved dependency tree and versions to support reproducible installs.

16. Difference between __dirname, process.cwd(), and import.meta.url?

__dirname: Directory of the current file in CommonJS.

process.cwd(): Directory from which the Node.js process was started.

import.meta.url: URL of the current module in ES Modules.

17. What are environment variables, and how should you manage them?

Environment variables store configuration outside the source code, such as database URLs and API keys. Use process.env, validate required values at startup, and keep secrets in a secure secret manager rather than committing them to Git.

18. Difference between process.exit() and allowing the event loop to finish naturally?

process.exit() terminates the process immediately and can interrupt pending asynchronous work. Allowing the event loop to finish lets pending work complete; graceful shutdown should stop accepting requests and close resources first.

19. How does Node.js handle concurrent requests?

Node.js uses non-blocking I/O to manage many requests concurrently. While one request waits for a database or network response, the main thread can process other work. CPU-heavy JavaScript can still block it.

20. What are streams and buffers in Node.js?

Buffer: Holds binary data in memory.

Stream: Processes data incrementally in chunks instead of loading everything at once.

Streams are useful for large files, uploads, downloads, and processing data efficiently.

21. Is Node.js single-threaded? Explain with an example.

JavaScript executes on the main thread, but Node.js can handle many I/O operations concurrently.

const fs = require("fs");

```
console.log("Start");

fs.readFile("data.txt", "utf8", (err, data) => {
console.log("File read completed");
});

console.log("End");
```

Output:

Start

End

File read completed

Interview answer: Node.js uses a single main JavaScript thread with non-blocking I/O, allowing it to handle multiple requests without waiting for each I/O operation to finish.

22. Explain the event loop.

The event loop coordinates when asynchronous callbacks can execute.

JavaScript call stack

Asynchronous operations

File system · Network · Timers

Event loop schedules callbacks

Callbacks execute on the main thread

Interview answer: The event loop allows Node.js to execute callbacks when asynchronous operations are ready, without blocking JavaScript while waiting for I/O.

23. Explain process.nextTick(), Promise, setTimeout(), and setImmediate().

```
console.log("A");

process.nextTick(() => console.log("B"));

Promise.resolve().then(() => console.log("C"));

setTimeout(() => console.log("D"), 0);

setImmediate(() => console.log("E"));

console.log("F");
```

The output begins with:

A

F

B

C

D and E can appear in either order when scheduled from the main module.

Interview answer: Synchronous code runs first. Node.js then processes the nextTick queue and Promise microtasks before continuing through event-loop phases. The relative ordering of the timer and immediate is context-dependent.

24. What happens when CPU-intensive work runs in Node.js?

```
app.get("/heavy", (req, res) => {
let total = 0;
```

```
for (let i = 0; i < 1e9; i++) {
total += i;
}

res.json({ total });
});
```

This loop blocks the main JavaScript thread. Other requests handled by that thread must wait.

Solution: Use worker threads for CPU-intensive JavaScript or offload the work to a separate service or background job.

25. What is the difference between blocking and non-blocking code?

```
const fs = require("fs");

// Blocking
const data = fs.readFileSync("data.txt", "utf8");

// Non-blocking
fs.readFile("data.txt", "utf8", (err, data) => {
if (err) return console.error(err);
console.log(data);
});
```

Interview answer: Blocking code makes the current JavaScript execution wait. Non-blocking I/O allows JavaScript to continue while the operation is pending.

26. What is the difference between Promise.all() and Promise.allSettled()?

```
const results = await Promise.all([
fetchUsers(),
fetchProducts(),
]);
```

Promise.all() rejects when one input Promise rejects.

Promise.allSettled() waits for every input to settle and returns each result's status.

When to use: Use Promise.all() when every result is required; use Promise.allSettled() when you need to process successes and failures independently.

27. How do you handle errors in Express?

```
app.get("/users", async (req, res, next) => {
try {
const users = await User.find();
res.json(users);
} catch (error) {
next(error);
}
});

app.use((err, req, res, next) => {
console.error(err);
```

```
res.status(500).json({
message: "Internal server error",
});
});
```

Interview answer: I use centralized error-handling middleware to keep error responses consistent, avoid duplicating logic, and prevent internal details from leaking to clients.

28. What is the difference between CommonJS and ES Modules?

CommonJS:

```
const express = require("express");
module.exports = app;
```

ES Modules:

```
import express from "express";
export default app;
```

Interview answer: CommonJS uses require and module.exports; ES Modules use static import and export syntax. Node.js supports both module systems.

29. What are streams and buffers?

```
const fs = require("fs");

const stream = fs.createReadStream("large-file.txt");

stream.on("data", (chunk) => {
console.log("Received chunk:", chunk.length);
});
```

Interview answer: A buffer stores binary data in memory, whereas a stream processes data incrementally. Streams reduce memory usage when handling large files.

30. How does Node.js handle thousands of concurrent requests?

Interview answer: Node.js uses non-blocking I/O so requests waiting on network or database operations do not need to occupy the main JavaScript thread. For higher throughput, I would optimize database queries, limit concurrency, use connection pooling and caching, and scale across processes or instances as needed.

Remember: high concurrency does not mean unlimited capacity. CPU-heavy work, memory consumption, and database connection limits can still become bottlenecks.

21. What is callback-based programming, and what is callback hell?

A callback is a function passed to another function to execute later. Callback hell occurs when callbacks are deeply nested, making code difficult to read and maintain.

```
getUser((user) => {
getOrders(user.id, (orders) => {
console.log(orders);
});
});
```

Solution: Use Promises or async/await.

22. What are Promises, and what are their possible states?

A Promise represents the eventual result of an asynchronous operation.

Pending: Operation is in progress.

Fulfilled: Operation succeeded.

Rejected: Operation failed.

A settled Promise cannot change its state again.

23. Explain async and await conceptually.

async makes a function return a Promise. await pauses that async function until a Promise settles, without blocking the main JavaScript thread.

async function getUser() {

const user = await fetchUser();

return user;

}

24. What happens when an awaited Promise rejects?

The await expression throws the rejection reason. Handle it with try/catch; otherwise, the async function's returned Promise rejects.

try {

const data = await fetchData();

} catch (error) {

console.error(error);

}

25. Difference between Promise.all(), allSettled(), race(), and any()?

Promise.all(): Fulfills when all fulfill; rejects if any rejects.

Promise.allSettled(): Waits for all to settle.

Promise.race(): Settles with the first Promise to settle.

Promise.any(): Fulfills with the first fulfillment; rejects if all reject.

26. When should you use Promise.all() instead of sequential await?

Use Promise.all() when operations are independent and can run concurrently.

const [users, products] = await Promise.all([

getUsers(),

getProducts(),

]);

Sequential await is necessary when one operation depends on the previous result.

27. How do you handle errors in asynchronous functions?

Use try/catch with async/await, or .catch() with Promise chains. Propagate errors to centralized error handling when appropriate.

28. What is an unhandled Promise rejection?

It occurs when a Promise rejects without an appropriate rejection handler. It can cause warnings or process termination, depending on Node.js configuration and version.

Best practice: Handle rejections explicitly; don't rely on process-level handlers as a substitute.

29. Difference between operational errors and programmer errors?

Operational errors: Expected runtime failures, such as a database timeout or unavailable service.

Programmer errors: Bugs, such as accessing an undefined property or using incorrect logic.

Handle operational errors appropriately and fix programmer errors rather than hiding them.

30. How would you implement centralized error handling?

Pass errors to Express error middleware using next(error). The middleware logs the error and returns a consistent, safe HTTP response.

```
app.use((err, req, res, next) => {
console.error(err);

res.status(err.statusCode || 500).json({
message: err.statusCode ? err.message : "Internal server error",
});
});
```

31. How do you create a custom error class in Node.js?

Extend the built-in Error class to attach additional information, such as an HTTP status code.

```
class AppError extends Error {
constructor(message, statusCode) {
super(message);
this.statusCode = statusCode;
}
}

throw new AppError("User not found", 404);
```

32. Difference between throwing an error and returning a rejected Promise?

throw raises an exception. Promise.reject() returns a rejected Promise.

Inside an async function, throwing an error causes the function's returned Promise to reject.

33. How do you prevent one failed operation from crashing unrelated work?

Handle failures at the appropriate task boundary. Use Promise.allSettled() when independent tasks should finish regardless of individual failures, and isolate errors in background jobs.

34. How do you handle errors from asynchronous callbacks?

Check the callback's error argument and handle or propagate it.

```
fs.readFile("data.txt", (err, data) => {
if (err) {
console.error(err);
return;
}

console.log(data.toString());
});
```

A surrounding try/catch does not automatically catch errors thrown later inside an asynchronous callback.

35. What are try, catch, and finally?

try: Executes code that might fail.

catch: Handles a thrown error.

finally: Runs cleanup code whether the operation succeeds or fails.

Use finally to release resources or reset state, such as clearing a loading indicator.

36. How do you implement a timeout for an API call?

Use AbortSignal.timeout() with Node.js's built-in fetch().

```
const response = await fetch(url, {
signal: AbortSignal.timeout(5000),
});
```

This requests cancellation after five seconds. The operation must support the signal for cancellation to work.

37. What is retry logic, and why should retries use backoff?

Retry logic repeats a failed operation when the failure may be temporary. Backoff delays retries to avoid overwhelming an already struggling service.

Retry only appropriate failures, and consider idempotency before retrying writes.

38. What is exponential backoff with jitter?

Exponential backoff increases the delay between retries; jitter adds randomness to those delays.

For example, retries may use a randomized delay within a progressively increasing time window. This prevents many clients from retrying simultaneously.

39. How do you limit concurrent asynchronous tasks?

Use a concurrency limiter or worker queue to cap the number of tasks running simultaneously.

For example, process 100 API requests with a maximum of 5 active requests at a time, rather than launching all 100 concurrently.

40. How would you debug an asynchronous race condition in Node.js?

A race condition occurs when the outcome depends on the timing or ordering of concurrent operations.

Approach: Reproduce the issue, add structured logs and correlation IDs, inspect shared state and database updates, and use atomic operations, transactions, locks, or optimistic concurrency control where appropriate.

Quick revision: remember these five distinctions

41. Explain Promise.all() vs Promise.allSettled()

```
const getUser = () => Promise.resolve("User");
const getProducts = () => Promise.reject(new Error("API failed"));

const results = await Promise.allSettled([
getUser(),
getProducts(),
]);

console.log(results);
```

Output (simplified):

```
[
{ status: "fulfilled", value: "User" },
{ status: "rejected", reason: Error("API failed") }
]
```

Interview answer: Promise.all() rejects if any input rejects. Promise.allSettled() waits for every input to settle and returns each result's status. I use Promise.all() when every result is required and allSettled() when independent operations should be handled separately.

42. When should you use Promise.all()?

Imagine a dashboard that needs users and products from independent APIs.

```
async function loadDashboard() {
const [users, products] = await Promise.all([
getUsers(),
getProducts(),
]);

return { users, products };
}
```

Both operations start without waiting for the other to finish.

Interview answer: I use Promise.all() when operations are independent and their results are all required. If one operation depends on another, I use sequential await.

43. How do you implement centralized error handling in Express?

```
app.get("/users/:id", async (req, res, next) => {
try {
const user = await User.findById(req.params.id);

if (!user) {
return res.status(404).json({ message: "User not found" });
}

res.json(user);
} catch (error) {
next(error);
}
});

// Register after routes
app.use((err, req, res, next) => {
console.error(err);

res.status(err.statusCode || 500).json({
message: err.statusCode ? err.message : "Internal server error",
});
});
```

Interview answer: I forward errors to centralized Express error middleware, which logs failures and returns consistent HTTP responses. I avoid exposing stack traces or sensitive details to clients. In Express 5, rejected Promises returned by route handlers are automatically forwarded to error handling; Express 4 generally requires explicit forwarding or an async wrapper.

44. How do you implement an API timeout?

```
async function fetchUsers() {
const response = await fetch(
"https://api.example.com/users",
{ signal: AbortSignal.timeout(5000) }
);
```

```
if (!response.ok) {
throw new Error(`HTTP error: ${response.status}`);
}

return response.json();
}
```

Interview answer: I set a five-second timeout using AbortSignal.timeout(). If the request exceeds the timeout, the fetch is aborted. I also handle timeout errors and distinguish them from other failures.

45. How do you debug an asynchronous race condition?

Suppose two requests update the same account balance using a read-modify-write sequence.

```
const account = await Account.findById(id);

account.balance -= 100;
```

Section 3: Express.js and Middleware — Interview Answers (Q41–60)

Q46. What is Express.js, and why is it used with Node.js?

Express.js is a lightweight web framework for Node.js that simplifies building APIs, handling routes, middleware, and HTTP requests.

Q42. What is middleware in Express?

Middleware is a function that runs during the request-response cycle. It can access req, res, and next to process requests, send responses, or pass control.

Q47. What is the difference between application-level, router-level, built-in, and third-party middleware?

Application-level: Registered using app.use() or app.METHOD().

Router-level: Registered on an express.Router() instance.

Built-in: Provided by Express, such as express.json().

Third-party: Installed separately, such as cors or morgan.

Q48. What is the difference between app.use() and app.get()?

app.use() registers middleware, generally for all HTTP methods under a path. app.get() handles only HTTP GET requests for a matching route.

Q45. What is the purpose of express.json()?

It parses incoming JSON request bodies and makes the parsed data available in req.body.

Q49. Explain the Express request-response lifecycle.

A request enters middleware in registration order, reaches a matching route, executes business logic, and returns a response. Errors can be forwarded to error-handling middleware.

Q50. How does middleware execution order affect application behavior?

Middleware runs in registration order. Incorrect ordering can cause missing request data, skipped authentication, or unreachable routes.

Q51. What is next(), and what happens if you forget to call it?

next() passes control to the next middleware or route handler. If you neither call it nor send a response, the request may hang.

Q52. What is next('route'), and when would you use it?

It skips the remaining callbacks of the current route and moves to the next matching route. Use it when a route condition is not satisfied.

Q53. How does Express error-handling middleware differ from normal middleware?

Normal middleware processes requests; error middleware handles errors and typically sends an appropriate error response or forwards the error.

Q54. Why does Express error middleware typically have four parameters?

Express identifies error middleware by its signature: (err, req, res, next). The first parameter contains the error.

Q55. How do you handle rejected Promises in Express 4 versus Express 5?

In Express 4, catch rejected Promises and call next(err). Express 5 automatically forwards rejected Promises returned by route handlers and middleware to error handling.

Q56. What is express.Router(), and how does it help organize a large application?

express.Router() creates modular route handlers, allowing routes to be separated by feature, such as users, products, and orders.

Q57. How do route parameters, query parameters, and request body data differ?

Route parameters: Identify resources, e.g. /users/:id → req.params.id.

Query parameters: Filter or paginate, e.g. ?page=2 → req.query.page.

Body: Contains submitted data → req.body.

Q58. How would you implement request validation in Express?

Validate request parameters, query values, and body data using a schema library such as Zod or Joi before business logic or database operations.

Q59. How would you build reusable authentication middleware?

Extract the authentication token, verify it, attach trusted user information to req, and call next(). Return 401 if authentication fails.

Q60. How would you implement role-based authorization middleware?

Check whether the authenticated user's role has the required permission. Return 403 if access is forbidden.

Q61. How do you handle file uploads in Express?

Use middleware such as Multer to process multipart uploads. Validate file type and size, generate safe filenames, and store files securely.

Q62. How would you implement request logging, correlation IDs, and centralized error responses?

Use logging middleware to record request details and duration, assign a correlation ID to trace requests, and add centralized error middleware for consistent responses. Never log secrets.

Q63. How would you gracefully shut down an Express server without dropping in-flight requests unnecessarily?

Handle SIGTERM and SIGINT, stop accepting new connections with server.close(), allow active requests to finish, close database connections, and enforce a shutdown timeout.

await account.save();

Two requests may read the same initial balance and overwrite each other's updates.

A safer approach for a simple decrement:

const account = await Account.findOneAndUpdate(

{ _id: id, balance: { $gte: 100 } },

{ $inc: { balance: -100 } },

{ new: true }

);

if (!account) {

throw new Error("Account not found or insufficient balance");

}

The conditional atomic update prevents concurrent requests from both spending the same available funds through this operation. Real financial systems may also need transactions, idempotency, and ledger-based accounting.

Interview answer: I reproduce the issue, trace requests using structured logs, and identify shared-state updates. I then use atomic database operations, optimistic concurrency, or transactions depending on the consistency requirements.

Section 4: REST APIs and HTTP (Q61–80)

Q61. What is REST, and what makes an API RESTful?

REST is an architectural style for designing APIs around resources, using standard HTTP methods and stateless communication.

Q62. What is the difference between GET, POST, PUT, PATCH, and DELETE?

GET: Retrieve data.

POST: Create a resource or trigger an operation.

PUT: Replace a resource.

PATCH: Partially update a resource.

DELETE: Delete a resource.

Q63. What are safe and idempotent HTTP methods?

Safe methods don't intend to modify server state. Idempotent methods produce the same intended effect when repeated. GET is safe and idempotent; PUT and DELETE are idempotent but not safe.

Q64. Explain common HTTP status codes.

200: Success.

201: Resource created.

204: Success, no response body.

400: Invalid request.

401: Authentication required or invalid.

403: Access forbidden.

404: Resource not found.

409: Conflict.

422: Request understood but validation or semantic rules failed.

429: Too many requests.

500: Internal server error.

Q65. What is the difference between 401 and 403?

401 means valid authentication is missing or unavailable. 403 means the user is authenticated but lacks permission.

Q66. When should an API return 400 versus 422?

Use 400 for malformed or invalid requests. Use 422 when the request is syntactically valid but fails validation or business rules. APIs may define their own consistent convention.

Q67. How do you design consistent API response formats?

Use a consistent structure for data, errors, and pagination.

{ "success": true, "data": {} }

For errors, include a clear message and an appropriate status code.

Q68. How would you implement pagination using limit/offset and cursor-based pagination?

Limit/offset: Use limit and skip; simple but can become slow for large offsets.

Cursor-based: Use a stable field such as _id to fetch the next page; more efficient for large datasets.

Q69. What are filtering, sorting, and searching in REST APIs?

Filtering: Select records by criteria.

Sorting: Order records by a field.

Searching: Find records matching a search term.

Example: /products?category=books&sort=price&search=node

Q70. How do you version REST APIs?

Use versioning such as /api/v1/users. Maintain backward compatibility and communicate deprecation before removing older versions.

Q71. What is CORS, and why can a React frontend encounter CORS errors?

CORS is a browser security mechanism controlling cross-origin requests. A React app may fail when the backend doesn't allow its frontend origin, methods, or headers.

Q72. What are preflight requests?

A preflight is an OPTIONS request the browser sends before certain cross-origin requests to check whether the server permits the requested method and headers.

Q73. What is the difference between cookies, local storage, and session storage?

Cookies: Can be sent automatically with HTTP requests; support HttpOnly, Secure, and SameSite.

Local storage: Persists in the browser until cleared.

Session storage: Usually lasts for the current browser tab's session.

Avoid storing sensitive tokens in browser storage when safer alternatives are appropriate.

Q74. What are HTTP headers, and which are important for API security?

Headers carry metadata about requests and responses. Important examples include Authorization, Content-Type, Origin, Cache-Control, and security headers such as Content-Security-Policy and Strict-Transport-Security.

Q75. What is content negotiation?

Content negotiation lets a client and server agree on a representation format using headers such as Accept and Content-Type.

Q76. What is API idempotency, and how do you prevent duplicate payments or orders?

Use an idempotency key for each logical operation. Store the key and result atomically so retries return the original result instead of creating duplicate transactions.

Q77. How do you handle large API responses efficiently?

Use pagination, filtering, field selection, compression, appropriate indexes, and streaming for suitable large data transfers.

Q78. What is rate limiting, and how would you implement it?

Rate limiting restricts requests per client over a time window. Use middleware or a distributed store such as Redis to enforce limits across multiple server instances.

Q79. How do you keep APIs backward-compatible?

Add optional fields, preserve existing response contracts, avoid changing existing field meanings, and deprecate old behavior gradually.

Q80. How would you investigate an API that works in Postman but fails in a browser?

Check the browser Network tab and Console for CORS errors, preflight failures, missing headers, cookie settings, authentication differences, and mixed-content issues.

Section 5: MongoDB Fundamentals (Q81–100)

Q81. What is MongoDB, and how does it differ from PostgreSQL or MySQL?

MongoDB is a document-oriented NoSQL database storing BSON documents. Relational databases primarily store data in tables with rows and relationships.

Q82. What are databases, collections, and documents?

Database: Contains collections.

Collection: Contains documents.

Document: A record represented as field-value pairs.

Q83. What is BSON, and how does it differ from JSON?

BSON is a binary document format used by MongoDB. It supports additional data types, such as ObjectId, dates, and binary data.

Q84. What is the difference between embedding and referencing?

Embedding stores related data inside a document. Referencing stores identifiers pointing to separate documents.

Q85. When would you choose embedded documents?

Use embedding when related data is frequently read together, has a manageable size, and benefits from atomic updates within one document.

Q86. What are the advantages and disadvantages of MongoDB schema flexibility?

It supports evolving data structures and faster iteration, but inconsistent documents can complicate validation, queries, and maintenance.

Q87. What is an ObjectId?

ObjectId is a commonly used MongoDB identifier, typically containing a timestamp component and other unique components.

Q88. What is the difference between find(), findOne(), and findById() in Mongoose?

find(): Returns an array of matching documents.

findOne(): Returns the first matching document or null.

findById(): Finds a document by its _id.

Q89. How do you perform CRUD operations in MongoDB?

Create: create() or insertMany()

Read: find() or findOne()

Update: updateOne() or findByIdAndUpdate()

Delete: deleteOne() or findByIdAndDelete()

Q90. Explain $set, $unset, $inc, $push, $pull, and $addToSet.

$set: Set a field.

$unset: Remove a field.

$inc: Increment a numeric value.

$push: Append an array element.

$pull: Remove matching array elements.

$addToSet: Add an array element only if an equivalent value isn't already present.

Q91. What is the difference between $push and $addToSet?

$push always appends an element. $addToSet prevents adding an equivalent element that already exists.

Q92. Explain $in, $nin, $and, $or, and $not.

$in: Matches any listed value.

$nin: Matches values not in the list.

$and: All conditions must match.

$or: At least one condition must match.

$not: Negates a specified query condition.

Q93. What is an index in MongoDB?

An index helps MongoDB find and sort documents more efficiently, reducing the need to scan the entire collection. Indexes consume storage and add write overhead.

Q94. Explain single-field, compound, unique, text, and multikey indexes.

Single-field: Indexes one field.

Compound: Indexes multiple fields.

Unique: Prevents duplicate indexed values.

Text: Supports text search.

Multikey: Indexes array values.

Q95. What is a compound index, and why does field order matter?

A compound index covers multiple fields. Field order affects which queries and sort operations can efficiently use the index, following the index-prefix rule.

Q96. What is a covered query?

A covered query can return all required fields directly from an index without fetching the full documents, potentially improving performance.

Q97. What is the MongoDB aggregation pipeline?

It processes documents through sequential stages to filter, group, transform, sort, and analyze data.

Q98. Explain common aggregation stages.

$match: Filter documents.

$group: Group and calculate values.

$project: Select or transform fields.

$sort: Sort documents.

$limit: Limit results.

$unwind: Expand array elements into separate documents.

$lookup: Join data from another collection.

Q99. How would you calculate total orders and revenue for each customer?

Assuming each order contains customerId and total:

```
db.orders.aggregate([
{
$group: {
_id: "$customerId",
totalOrders: { $sum: 1 },
totalRevenue: { $sum: "$total" }
}
}
]);
```

Q100. How would you optimize a slow MongoDB query?

Use explain("executionStats"), check indexes, reduce unnecessary documents and fields, optimize sorting and aggregation stages, and inspect query execution statistics.

Section 6: Mongoose and Data Modeling (Q101–120)

Q101. What is Mongoose?

Mongoose is an ODM for Node.js that provides schemas, models, validation, middleware, and convenient MongoDB operations.

Q102. What is the difference between a schema and a model?

A schema defines document structure and rules. A model uses that schema to create, query, update, and delete documents.

Q103. What is the difference between schema-level validation and API request validation?

API validation checks incoming client data at the application boundary. Mongoose validation checks document data during supported Mongoose operations. Both are useful.

Q104. How do you define required fields, enums, defaults, and custom validators?

const userSchema = new mongoose.Schema({

name: { type: String, required: true },

role: { type: String, enum: ["user", "admin"], default: "user" },

age: { type: Number, validate: v => v >= 18 }

});

Q105. What is the difference between findByIdAndUpdate() and findOneAndUpdate()?

findByIdAndUpdate(id, update) updates by _id; findOneAndUpdate(filter, update) accepts a general filter.

Q106. Why use { new: true, runValidators: true }?

new: true returns the updated document. runValidators: true runs applicable schema validators during the update.

Q107. What does .populate() do, and what are its trade-offs?

.populate() replaces referenced IDs with related documents. It simplifies reads but can add database work, latency, and memory usage.

Q108. What is the difference between .lean() and ordinary Mongoose queries?

.lean() returns plain JavaScript objects instead of full Mongoose documents, reducing overhead when document methods and change tracking aren't needed.

Q109. What are Mongoose middleware hooks?

pre() hooks run before an operation; post() hooks run afterward. They support tasks such as normalization, auditing, and lifecycle logic.

Q110. What is the difference between document, query, and model middleware?

Document middleware: Runs on document operations, such as save.

Query middleware: Runs on queries, such as find.

Model middleware: Runs on model-level operations, such as insertMany.

Q111. How would you implement soft deletion?

Add a field such as deletedAt. Set it when deleting and exclude soft-deleted records from normal queries.

Q112. What are virtual properties in Mongoose?

Virtuals are computed properties that aren't stored in MongoDB, such as a user's full name derived from first and last names.

Q113. What are Mongoose instance methods and static methods?

Instance methods operate on individual documents; static methods operate on the model or collection.

Q114. How do you prevent duplicate records using unique indexes?

Create a unique index on the relevant field or field combination. Handle duplicate-key errors, typically MongoDB error code 11000.

Q115. Why is unique: true not a normal validation rule?

It creates a unique index; it doesn't perform ordinary Mongoose validation. Duplicate values are rejected by MongoDB, producing a duplicate-key error.

Q116. What is a MongoDB transaction, and when would you use one?

A transaction groups multiple database operations so they commit together or roll back together. Use it when related changes across documents or collections must be atomic.

Q117. What are MongoDB sessions?

Sessions group database operations and provide the context needed for transactions. Operations must use the same session to participate in that transaction.

Q118. What are optimistic concurrency control and __v?

Optimistic concurrency detects conflicting updates without locking documents in advance. Mongoose's __v is a version key; optimistic concurrency can use it to detect conflicting document saves when configured.

Q119. How do you model one-to-many and many-to-many relationships?

Use embedding or references for one-to-many, depending on access patterns and data size. Many-to-many relationships commonly use references or a separate linking collection.

Q120. How would you design schemas for users, products, orders, and order items?

Use separate User, Product, and Order collections. Store the customer reference and an array of order items containing product references, quantity, and price at purchase time so historical orders remain accurate.

Concept	Key point
async/await	Cleaner syntax for Promise-based code
Promise.all()	Concurrent work; fails fast on rejection
Promise.allSettled()	Collects every task's outcome
throw inside async	Rejects the returned Promise
Backoff + jitter	Reduces retry storms